

Site Features, Roles, Flows, and Certificate Architecture Guide

US-English documentation for the current WebAppExperimental266 site structure, page-to-asset relationships, role activation paths, certificate workflows, YubiKey login handling, and automated test behavior.

Site Features, Roles, Flows, and Certificate Architecture Guide (en-US)

This guide documents the current structure of the site, the roles that are actually wired into the repository, the pages and assets those roles activate, the most common request flows, the certificate-authority processes behind privileged access, and how the automated tests interact with the application.

Current-scope note: The repository currently implements **public**, **authenticated user**, **site admin (admin certificate)**, **group admin (group-scoped certificate)**, and **YubiKey MFA policy** flows. The `/Experimental/*` YubiKey policy hook exists even though no Razor Pages currently live under that folder.

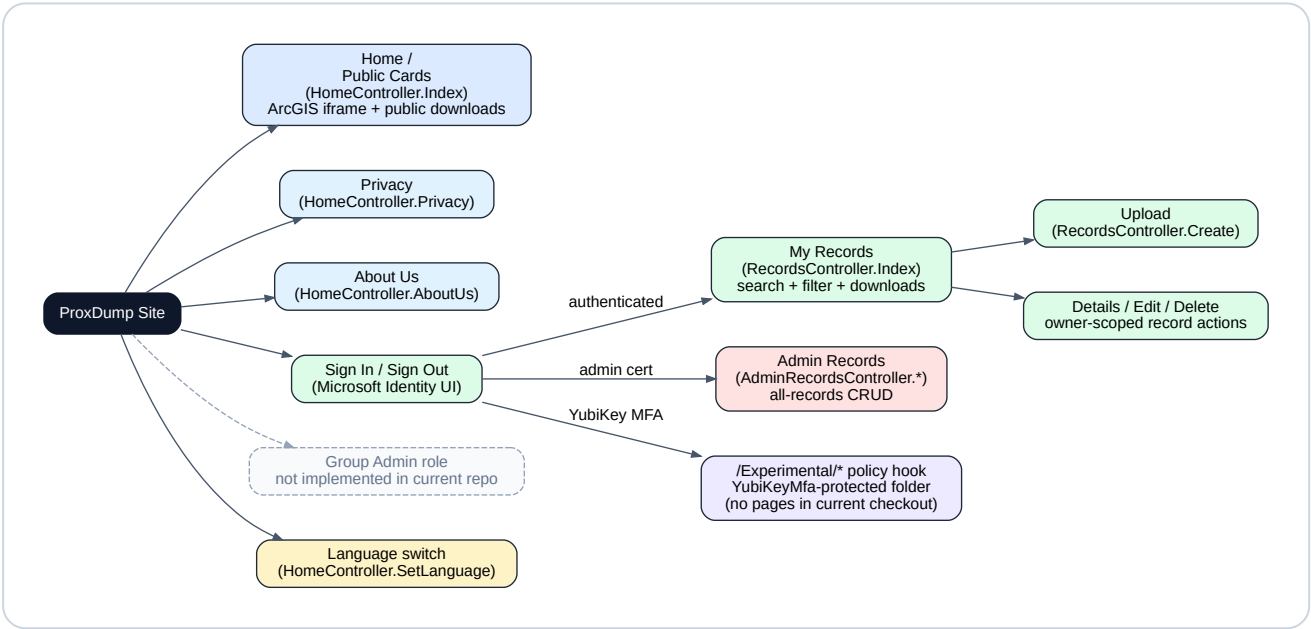
Feature summary

- Public visitors can browse public card dumps, open ArcGIS maps, switch language, and download public files.
- Authenticated users can sign in through an OIDC provider and manage their own records.
- Site admins must pass the normal login flow **and** present a mapped admin client certificate to access the admin-records surface.
- Group admins must pass the normal login flow **and** present a mapped group-admin client certificate to access group-scoped records.
- YubiKey MFA is modeled as a second factor for `/Experimental/*`: a signed-in user must also present a trusted YubiKey PIV certificate and pass OCSP validation.
- The app can integrate with Azure Key Vault, Cosmos DB, Blob Storage, AWS services, and GCP services through feature flags.
- Tests are run with `dotnet test`; they are not executed during plain compilation.

Role inventory

Role	Implemented now	Gate	Main active pages	Main assets/services activated
Public visitor	Yes	None	Home, Privacy, About, public downloads	ArcGIS public URL, CRUD read for public rows, static assets
Authenticated user	Yes	OIDC login (Azure AD, AWS Cognito, or GCP Identity)	My Records, Upload, owner Details/Edit/Delete	Auth cookie, CRUD owner queries, inline filters, user ArcGIS URL
Site admin	Yes	OIDC login + AdminCertificate policy	Admin Records CRUD	Client certificate, audit logging, full CRUD, admin ArcGIS URL
YubiKey MFA user	Policy only	OIDC login + YubiKeyMfa policy	/Experimental/* if pages are added	YubiKey PIV certificate, OCSP, optional attestation checks
Group admin	Yes	OIDC login + GroupAdminCertificate policy	Group Admin Records CRUD	Client certificate mapped to group/user identity, group-scoped CRUD, audit logging

Diagram 1 — whole-site structure



The site surface is centered on four implemented controller areas:

- `HomeController` for public content and role-based map selection.
- `RecordsController` for owner-scoped CRUD actions after sign-in.
- `AdminRecordsController` for admin-scoped CRUD actions after both primary login and certificate validation.
- `GroupAdminRecordsController` for group-scoped CRUD actions after both primary login and group-admin certificate validation.

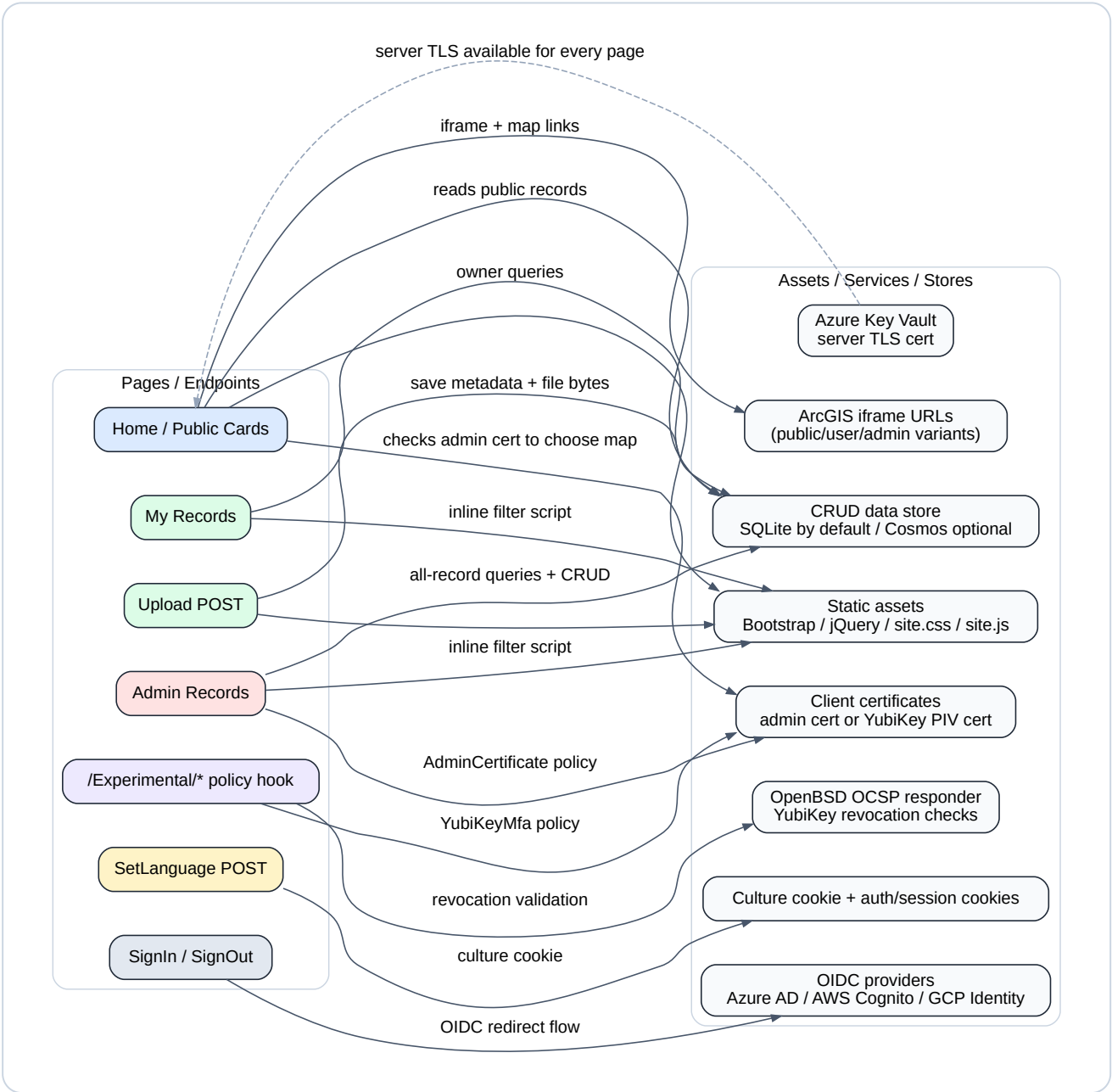
The repo also wires a YubiKey policy to `/Experimental/*`, but there are no Razor Pages under `Pages/Experimental` in this checkout.

Page and route inventory

Route or surface	Access	Core code path	What it does
<code>/</code>	Public	<code>HomeController.Index</code>	Lists public records and chooses a public/user/admin ArcGIS map URL
<code>/Privacy</code> , <code>/Home/Privacy</code>	Public	<code>HomeController.Privacy</code>	Static privacy page
<code>/AboutUs</code> , <code>/Home/AboutUs</code>	Public	<code>HomeController.AboutUs</code>	Static about page

Route or surface	Access	Core code path	What it does
POST /Home/SetLanguage	Public	HomeController.SetLanguage	Persists culture choice in a cookie
/Records	Authenticated user	RecordsController.Index	Lists the current user's records
/upload	Authenticated user	RecordsController.Create	Uploads a JSON card dump and metadata
/Records/Details/{id}	Owner/public read rules	RecordsController.Details	Shows one record
/Records/Edit/{id}	Owner only	RecordsController.Edit	Updates title/description
/Records/Delete/{id}	Owner only	RecordsController.Delete	Removes a record
/Records/Download/{id}	Public or authorized	RecordsController.Download	Downloads file content
/AdminRecords/*	Site admin	AdminRecordsController.*	Full-record CRUD and admin search/filter
/GroupAdminRecords/*	Group admin	GroupAdminRecordsController.*	Group-scoped CRUD limited to the mapped group
/Account/SignIn , /Account/SignOut	Public/authenticated	Microsoft Identity UI	Starts or ends the primary login session
/Experimental/*	YubiKey MFA policy	Razor Pages convention	Reserved for second-factor protected pages

Diagram 2 — which pages interact with which assets

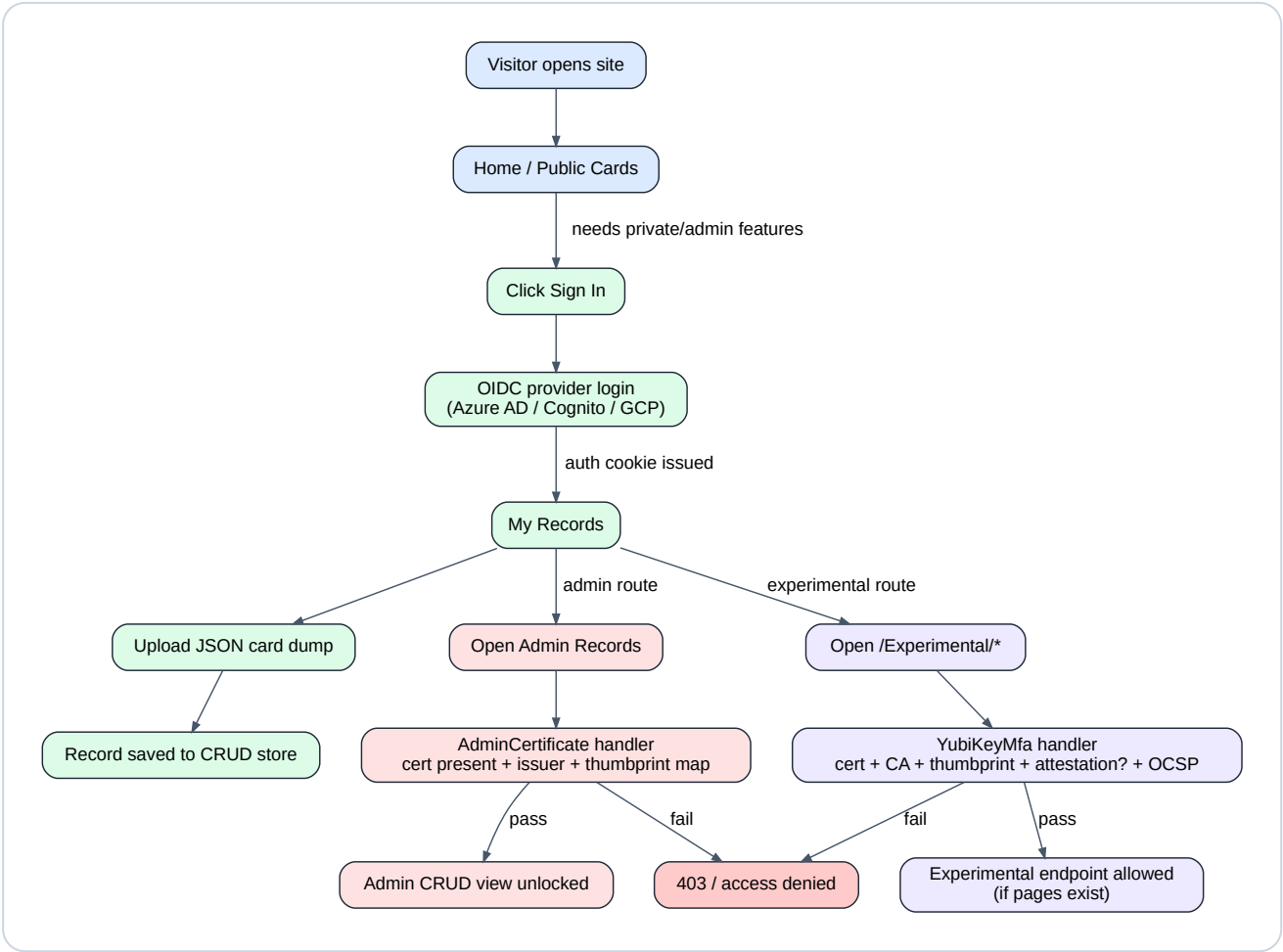


Key page-to-asset relationships:

Page	Asset or external dependency	Purpose
Home	ArcGIS iframe URL	Chooses public, user, or admin map based on auth and admin-certificate result
Home	CRUD store	Reads up to 50 public records
Records pages	CRUD store	Reads or writes owner-scoped records
Records/Admin views	Inline JavaScript + static assets	Client-side table filtering and layout
Admin Records	Client certificate + audit service	Unlocks admin surface and records access attempts

Page	Asset or external dependency	Purpose
Sign-in UI	OIDC provider	Establishes the primary identity
/Experimental/*	YubiKey certificate + OCSP responder	Enforces a second factor when the folder is used
Every HTTPS request	Azure Key Vault server certificate (optional feature)	Supplies the TLS server identity to Kestrel

Diagram 3 — common event flows

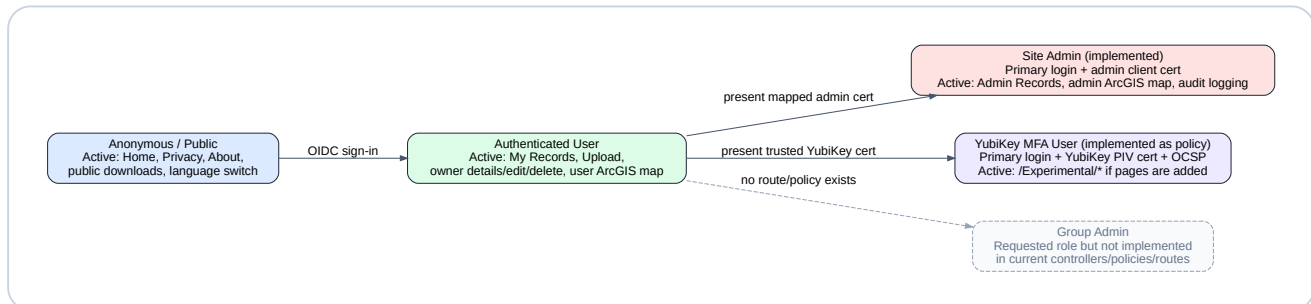


The most common user journeys are:

1. **Browse public content** → open the home page → see public records and the public ArcGIS map.
2. **Sign in and upload** → complete OIDC login → open My Records → upload a JSON file → write metadata and file bytes to the CRUD store.
3. **Enter the admin surface** → sign in → request /AdminRecords → pass AdminCertificateRequirementHandler → unlock full CRUD.

4. Enter a YubiKey-protected route → sign in → request `/Experimental/*` → pass YubiKey issuer, thumbprint, optional attestation, and OCSP checks.

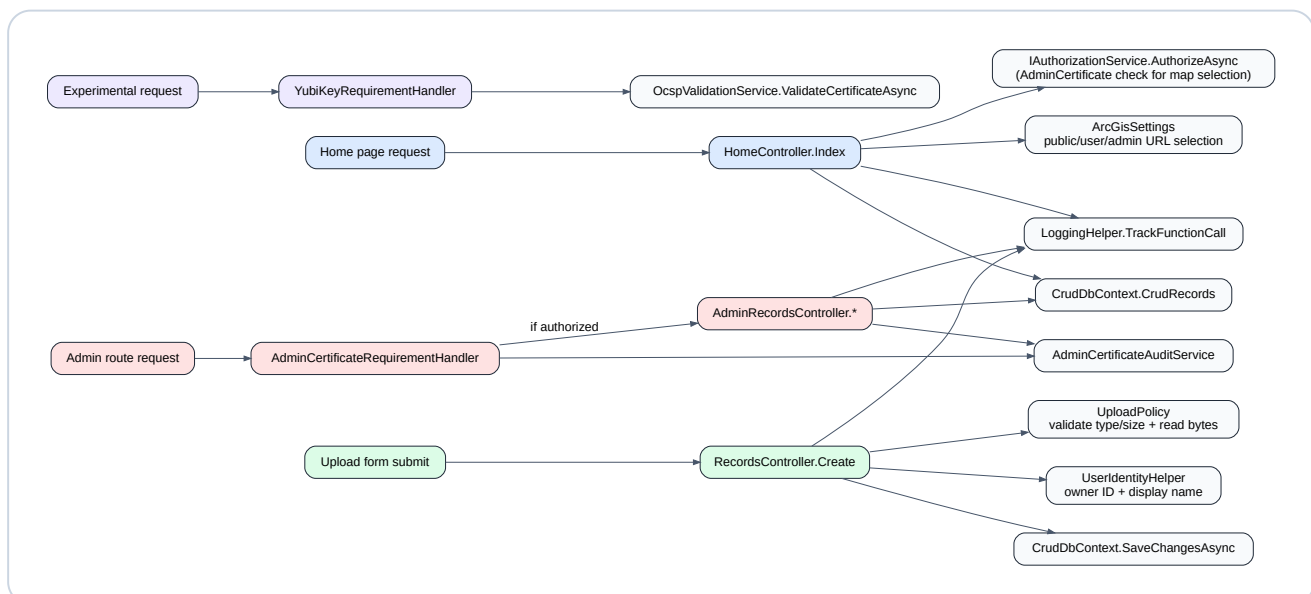
Diagram 4 — role activation state model



This role model reflects what the current code actually activates:

- **Anonymous** users only get public content.
- **Authenticated** users unlock owner-scoped CRUD pages.
- **Site admins** are a stricter state on top of authenticated users and require a mapped certificate.
- **Group admins** are a stricter state on top of authenticated users and require a mapped group-admin certificate.
- **YubiKey MFA users** are another stricter state on top of authenticated users and are intended for `/Experimental/*`.

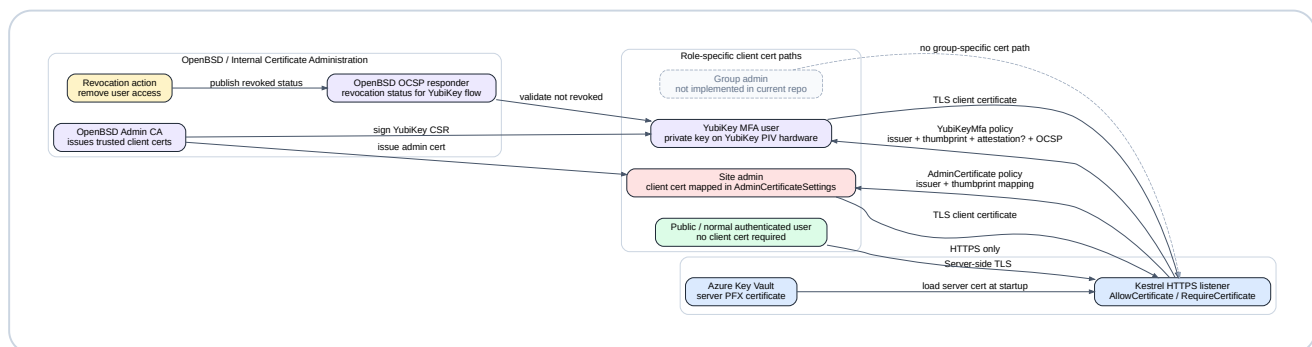
Diagram 5 — what programs and functions activate during key interactions



The main request paths activate these application components:

Interaction	Main controller/function path	Supporting services/functions
Open home page	<code>HomeController.Index</code>	<code>LoggingHelper.TrackFunctionCall</code> , <code>CrudDbContext.CrudRecords</code> , <code>IAuthorizationService.AuthorizeAsync</code> , <code>ArcGisSettings</code>
Upload record	<code>RecordsController.Create</code>	<code>UploadPolicy</code> , <code>UserIdentityHelper</code> , <code>CrudDbContext.SaveChangesAsync</code>
Browse own records	<code>RecordsController.Index</code>	<code>UserIdentityHelper.GetStableUserId</code> , <code>CrudDbContext</code>
Open admin pages	<code>AdminCertificateRequirementHandler</code> then <code>AdminRecordsController.*</code>	<code>AdminCertificateAuditService</code> , <code>CrudDbContext</code>
Open group-admin pages	<code>GroupAdminCertificateRequirementHandler</code> then <code>GroupAdminRecordsController.*</code>	<code>GroupAccessSettings</code> , <code>AdminCertificateAuditService</code> , <code>CrudDbContext</code>
Open YubiKey-protected route	<code>YubiKeyRequirementHandler</code>	<code>OcspValidationService</code> , <code>YubiKeySettings</code> , TLS client certificate chain
Client-side filtering	<code>Views/Records/Index.cshtml</code> and <code>Views/AdminRecords/Index.cshtml</code> inline scripts	Bootstrap, jQuery, nonce-bearing script tags when CSP is enabled

Diagram 6 — certificate authority and YubiKey workflow



The certificate architecture splits into two major trust paths:

Server certificate path

- The site can pull its **server TLS certificate** from **Azure Key Vault** at startup.
- Kestrel installs that server certificate for HTTPS.
- Every browser session depends on this server-side TLS identity.

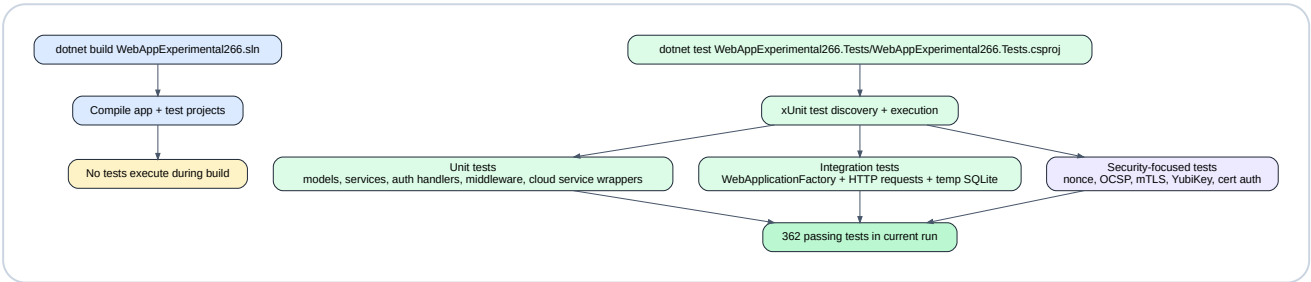
Privileged client-certificate path

- **Site admins** present a client certificate that must come from an allowed issuer and map to the signed-in admin's configured thumbprint.
- **YubiKey MFA users** present a client certificate backed by a private key generated on YubiKey PIV hardware.
- The **OpenBSD admin CA** can sign YubiKey CSRs and operate the **OCSP responder** used during YubiKey validation.
- Revocation is especially important for the YubiKey flow because the app checks OCSP status during protected requests.

Role-specific certificate expectations

Role	Client certificate required	Validation path
Public	No	Server TLS only
Authenticated user	No	OIDC login only
Site admin	Yes	<code>AdminCertificateSettings.AllowedIssuers</code> + mapped thumbprint
YubiKey MFA user	Yes	Allowed CA issuer + optional CA thumbprint + optional attestation + OCSP
Group admin	Yes	<code>GroupAccessSettings</code> group/user mapping + mapped thumbprint + allowed issuer

Diagram 7 — how tests interact with the site and when they run



The current repository behavior is:

- `dotnet build WebAppExperimental266.sln` compiles the application and test projects, but **does not run tests**.
- `dotnet test WebAppExperimental266.Tests/WebAppExperimental266.Tests.csproj` compiles as needed, discovers xUnit tests, and then runs them.
- Integration tests use `WebApplicationFactory<Program>` and a temporary SQLite database so they can send real HTTP requests through the app pipeline.

Test coverage map

Test area	What it exercises
Models tests	Configuration objects, defaults, validation helpers
Service tests	Auth handlers, OCSP logic, nonce services, cloud wrapper services
Extension tests	mTLS registration and pipeline ordering
Controller authorization tests	Required authorization attributes and policy usage
Integration tests	Home/Privacy routes, service registration, security headers, app startup
Security-focused tests	mTLS, YubiKey, admin certificate, nonce, OCSP behavior

Current timing answer

The automated tests run **during `dotnet test`** , not during a plain `dotnet build` . In the current validation run for this task, the repository passed **367 tests**.

Operational observations and gaps

- Group-admin-specific flows are implemented with a dedicated controller, views, authorization policy, certificate mapping model, and tests.
- `/Experimental/*` is already reserved for YubiKey MFA, so future feature work can attach new Razor Pages there without redesigning the auth model.
- The home page is the clearest page-to-asset hub because it combines database reads, authorization-sensitive ArcGIS selection, and public downloads.
- The admin views and user views both ship inline filter scripts, so CSP/nonce behavior matters for these pages when nonce services are enabled.

Security feature summary mapped to NIST controls (SP 800-53 Rev. 5)

Security feature area	Current implementation in this repo	NIST control mappings
Security headers	CSP with nonce/hash support; X-Frame-Options, X-Content-Type-Options, HSTS, Referrer-Policy, Permissions-Policy, COOP/CORP, cache-control hardening	SC-5 (Denial-of-Service protections), SC-7 (Boundary protection), SC-8 (Transmission confidentiality/integrity), SC-23 (Session authenticity), SI-10 (Information input validation)
Certificate-based access control	<code>AdminCertificate</code> and <code>GroupAdminCertificate</code> policies enforce mapped client-certificate access for privileged routes; failed/successful attempts are audited	IA-2 (Identification and authentication), IA-5 (Authenticator management), AC-3 (Access enforcement), AU-2/AU-12 (Auditable events and logging)
External resource use	Feature-flagged integrations for Azure Key Vault/Cosmos/Blob, AWS Secrets Manager/DynamoDB/Cognito, and GCP Secret Manager/Firestore/Identity with explicit startup wiring	CM-7 (Least functionality), SA-9 (External system services), SC-7 (Boundary protection), SR-3 (Supply chain controls)
MFA (YubiKey)	<code>YubiKeyMfa</code> policy enforces second factor via client cert checks, issuer/thumbprint constraints, optional attestation checks, and OCSP revocation validation	IA-2(1)/(2) (Multi-factor authentication), IA-5 (Authenticator management), SC-17 (PKI certificates), SI-4 (System monitoring through revocation/status checks)

Feature descriptions

The following sub-sections describe each major feature of the application as documented in the repository README.

Azure AD Authentication (OpenID Connect)

The application authenticates users through **Microsoft Identity Platform** using the OpenID Connect protocol (via `Microsoft.Identity.Web`). All routes under `/Experimental` require an authenticated Azure AD identity. The `/Privacy`, `/Error`, and `/About` pages are publicly accessible. The `[Authorize]` attribute on `HomeController` enforces authentication across all MVC actions.

Mutual TLS (mTLS) Client Certificate Authentication

When enabled, the application requires connecting clients to present a valid X.509 certificate. Settings in `MtlsSettings` control:

- Whether to allow chained certificates, self-signed certificates, or both
- Certificate revocation checking (X.509 CRL / online mode)
- Allowed certificate issuers (checked as a case-insensitive substring match against the certificate's `Issuer DN`)

The Kestrel web server is configured with `ClientCertificateMode.RequireCertificate` when mTLS is on, and `ClientCertificateMode.AllowCertificate` in development or when mTLS is off.

Azure Key Vault Integration

The application retrieves the TLS **server certificate** from Azure Key Vault at startup. The Key Vault client uses Azure AD client credentials (client ID + client secret) from configuration. The loaded `X509Certificate2` is injected directly into Kestrel's HTTPS defaults so no PFX file needs to exist on disk.

OCSP Certificate Revocation Validation

An `OcspValidationService` stub is included for validating client certificates against an OCSP (Online Certificate Status Protocol) server. The service supports configurable enable/disable, request timeout and retry count, in-memory caching of OCSP responses (configurable duration), and fail-closed, fail-open, or warn-only behavior when the OCSP server is unavailable.

Note: The actual OCSP wire-protocol implementation (`PerformOcspValidationAsync`) is a stub that rejects all certificates until a production implementation is supplied.

Content Security Policy with Per-Request Nonces

When enabled, every HTTP response carries a `Content-Security-Policy` header whose `script-src` directive includes a **cryptographically random nonce** generated per request. The nonce is:

1. Generated/refreshed by `NonceRefresherService` using AES-CBC encryption with a configurable 32-byte key and 16-byte IV stored in configuration (or User Secrets).
2. Catalogued in a thread-safe `ConcurrentDictionary` by `NonceCatalogService`.
3. Injected into every response by `NonceMiddleware` and placed in `HttpContext.Items["Nonce"]` so Razor views can embed it in `<script>` tags.

The CSP also supports SHA-256 hash-based allow-listing of inline scripts via a flat text file (`wwwroot/csp-hashes.txt`) and an optional manually specified hash in configuration.

Standard HTTP Security Headers

`UseStandardSecurityHeaders` appends the following headers to every response:

- `X-Frame-Options: DENY`
- `X-Content-Type-Options: nosniff`
- `Strict-Transport-Security: max-age=31536000; includeSubDomains`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Cross-Origin-Opener-Policy: same-origin`
- `Cross-Origin-Resource-Policy: same-site`
- `Permissions-Policy` disabling geolocation, camera, microphone, and FLoC (`interest-cohort`)
- Removal of `Server`, `X-Powered-By`, and `X-AspNetMvc-Version` response headers
- `Cache-Control: no-cache, no-store, must-revalidate`

Azure Blob Storage

When enabled, `BlobSettingsService` provides a scoped service backed by a connection string and a configurable maximum attachment count. The connection string is expected to be stored in User Secrets or Azure Key Vault, never in source control.

Azure Cosmos DB

When enabled, the application verifies the Cosmos DB connection at startup by calling `database.ReadAsync()`. `CosmosDbService` wraps a `CosmosClient` singleton and is bound to a configurable database and container. The connection string and account key are secrets stored outside source control.

AWS Secrets Manager

When enabled, `AwsSecretsManagerOperationsService` provides a template stub for fetching secrets and TLS certificates from **AWS Secrets Manager**. AWS credentials (`AccessKeyId` , `SecretAccessKey`) must be stored in User Secrets or environment variables — never in source control.

Amazon DynamoDB

When enabled, `AwsDynamoDbService` wraps an `IAmazonDynamoDB` client singleton. At startup the service verifies connectivity by calling `DescribeTable`. AWS credentials must be stored outside source control.

Google Cloud Secret Manager

When enabled, `GcpSecretManagerOperationsService` provides a template stub for fetching secrets and TLS certificates from **Google Cloud Secret Manager**. Authentication uses **Application Default Credentials (ADC)** by default; a service-account JSON key file path can optionally be supplied via `GcpSecretManager:CredentialFilePath`.

Google Cloud Firestore

When enabled, `GcpFirestoreService` wraps a `FirestoreDb` singleton. At startup the application builds a Firestore client bound to the configured project ID and collection name. Authentication uses ADC or a service-account JSON key file.

AWS Cognito Identity Management

When enabled, `AddAwsCognitoAuthentication` configures OpenID Connect authentication against an **AWS Cognito User Pool**. Configuration is under the `AwsCognito` section: `Region` , `UserPoolId` , `AppClientId` , `AppClientSecret` (store in User Secrets), and `Domain` . The callback path defaults to `/signin-aws-cognito` .

GCP Identity Platform

When enabled, `AddGcpIdentityAuthentication` configures OpenID Connect authentication using **Google's OAuth 2.0 / OpenID Connect** endpoint. Configuration is under the `GcpIdentity` section:

`ClientId` , `ClientSecret` (store in User Secrets), and optional `ProjectId` . The callback path defaults to `/signin-gcp` .

Secure Session Management

Sessions use in-process distributed memory cache with a **30-minute idle timeout**. Session cookies are configured as `HttpOnly = true` , `Secure = Always` (HTTPS-only), and `SameSite = Strict` .

Localization

The application supports **25 languages** with per-view `.resx` resource files. The active culture is determined at the request pipeline level via `RequestLocalizationOptions` (Accept-Language header, query string, or cookie). Users can switch language at any time using the language picker in the navigation bar. Right-to-left (RTL) layout is activated automatically when Arabic is selected.

Culture Tag	Language
en-US	English (United States) — default
de-DE	Deutsch (German)
es-ES	Español (Spanish)
fr-FR	Français (French)
pt-PT	Português (Portuguese)
it-IT	Italiano (Italian)
zh-HK	繁體 (Cantonese — Hong Kong Traditional Chinese)
ko-KR	한국어 (Korean)
hi-IN	हिन्दी (Hindi)
ru-RU	Русский (Russian)
ar-SA	العربية (Arabic — right-to-left layout)
sw-KE	Kiswahili (Swahili)
ja-JP	日本語 (Japanese)
ht-HT	Kreyòl ayisyen (Haitian Creole)
haw-US	‘Ōlelo Hawai‘i (Hawaiian)
sm-WS	Gagana Samoa (Samoan)
mi-NZ	Te Reo Māori (Māori)
af-ZA	Afrikaans

Culture Tag	Language
nĭ - NL	Nederlands (Dutch)
ha - NG	Hausa
am - ET	ፊንፋን (Amharic)
yo - NG	Yorùbá (Yoruba)
bn - BD	বাংলা (Bengali)
zh - CN	普通话 (Mandarin Chinese — Simplified)
ga - IE	Gaeilge (Irish)

PII-Safe Logging

`LoggingHelper` hashes personally identifiable information in log output using HMAC-SHA256. A stable 32-byte key can be supplied via `Logging:PiiHmacKey` (stored in User Secrets). If the key is absent or invalid, a random key is generated at startup so PII is never logged in plaintext.

Feature flags

All major subsystems are controlled by boolean feature flags in `appsettings.json`. Each flag defaults to a safe state.

Flag	Default	Description
<code>EnableSession</code>	<code>true</code>	Server-side session and session cookie
<code>EnableLocalization</code>	<code>true</code>	Multi-language support (25 languages)
<code>EnableAzureAd</code>	<code>true</code>	Azure AD / OpenID Connect authentication
<code>EnableAuthorization</code>	<code>true</code>	Route-level authorization policies
<code>EnableKeyVault</code>	<code>false</code>	Load TLS server cert from Azure Key Vault
<code>EnableNonceServices</code>	<code>false</code>	Per-request CSP nonce generation
<code>EnableCSP</code>	<code>false</code>	Attach <code>Content-Security-Policy</code> header
<code>EnableSecurityHeaders</code>	<code>true</code>	Attach standard HTTP security headers
<code>EnableBlobStorage</code>	<code>false</code>	Azure Blob Storage service
<code>EnableCosmosDb</code>	<code>false</code>	Azure Cosmos DB service
<code>EnableMtls</code>	<code>false</code>	Require client TLS certificates

Flag	Default	Description
<code>EnableOcsValidation</code>	<code>false</code>	OCSP certificate revocation check (stub)
<code>EnableAwsSecretsManager</code>	<code>false</code>	AWS Secrets Manager service (stub)
<code>EnableAwsDynamoDb</code>	<code>false</code>	Amazon DynamoDB service
<code>EnableAwsCognito</code>	<code>false</code>	AWS Cognito OpenID Connect identity management
<code>EnableGcpSecretManager</code>	<code>false</code>	GCP Secret Manager service (stub)
<code>EnableGcpFirestore</code>	<code>false</code>	Google Cloud Firestore service
<code>EnableGcpIdentity</code>	<code>false</code>	GCP Identity Platform (Google OAuth 2.0 / OIDC)
<code>EnableYubiKeyRequired</code>	<code>false</code>	YubiKey PIV MFA enforcement on <code>/Experimental/*</code>

Configuration reference

Copy `appsettings.template.json` to `appsettings.json` and replace all `{{PLACEHOLDER}}` values. Secrets must not be stored in source control — set them as App Service Application Settings, environment variables, or via .NET User Secrets locally.

Section	Key	Description
<code>AzureAd</code>	<code>TenantId</code> , <code>ClientId</code> , <code>ClientSecret</code>	Azure AD app registration
<code>AzureKeyVault</code>	<code>KeyVaultURL</code> , <code>KeyVaultSecret</code> , <code>KeyVaultPassName</code>	Key Vault and certificate name
<code>MtlsSettings</code>	<code>RequireClientCertificate</code> , <code>AllowedIssuers</code>	mTLS client cert policy
<code>NonceEncryption</code>	<code>Key</code> , <code>IV</code>	32-byte key and 16-byte IV for nonce encryption (base64)
<code>BlobSettings</code>	<code>BlobConnectionString</code> , <code>MaxAttachments</code>	Blob Storage connection
<code>CosmosDb</code>	<code>CosmosConnectionString</code> , <code>DatabaseName</code> , <code>ContainerName</code>	Cosmos DB connection
<code>OcsSettings</code>	<code>OcsServerUrl</code> , <code>CacheDurationMinutes</code>	OCSP validation (stub)

Section	Key	Description
Logging	PiiHmacKey	32-byte base64 HMAC key for PII hashing in logs
YubiKeySettings	AllowedIssuers , AllowedThumbprints , RequireAttestation	YubiKey PIV MFA policy
AdminCertificateSettings	AllowedIssuers , AdminUsers[*].UserIdentifiers , AdminUsers[*].AllowedThumbprints	Admin client certificate policy
GroupAccessSettings	Groups[*].GroupName , Groups[*].Users[*]	Group admin certificate mapping
AwsSecretsManager	Region , CertificateSecretName , AccessKeyId , SecretAccessKey	AWS Secrets Manager (stub)
AwsDynamoDb	Region , TableName , AccessKeyId , SecretAccessKey	Amazon DynamoDB
AwsCognito	Region , UserPoolId , AppClientId , AppClientSecret , Domain	AWS Cognito OIDC identity
GcpSecretManager	ProjectId , CertificateSecretId , CredentialFilePath	GCP Secret Manager (stub)
GcpFirestore	ProjectId , DatabaseId , CollectionName , CredentialFilePath	Google Cloud Firestore
GcpIdentity	ClientId , ClientSecret , ProjectId , CallbackPath	GCP Identity Platform (Google OAuth 2.0 / OIDC)

Supporting scripts

The `SupportingScripts/` directory contains PowerShell utilities:

Script	Purpose
<code>IvAndKeySampleGenerator.ps1</code>	Generate a random 32-byte AES key and 16-byte IV (base64)
<code>HashInlineScriptPowerShell.ps1</code>	Compute SHA-256 hashes for inline scripts (for CSP allow-listing)
<code>HashInlineScriptPowerShellBase64Output.ps1</code>	Same as above, outputs hashes in base64 format
<code>CertificateUploaderToAzureExample.ps1</code>	Upload a PFX certificate to Azure Key Vault

Script	Purpose
CheckRoles.ps1	Verify Azure RBAC role assignments for the app
ExportResourceGroups.ps1	Export Azure resource group configurations
TroubleshootingCosmosDBInfo.ps1	Diagnose Cosmos DB connectivity
SetupFromTemplate.ps1	Automate initial configuration from appsettings.template.json

Security notes

- **Never commit secrets** (ClientSecret , KeyVaultSecret , connection strings, encryption keys, AWS/GCP credentials) to source control. Use .NET User Secrets locally and Azure App Settings / Key Vault references in production.
- The OSCP validation implementation is a **stub** that rejects all certificates. Replace PerformOscpValidationAsync in OscpValidationService.cs before enabling EnableOscpValidation in production.
- The AWS Secrets Manager and GCP Secret Manager implementations are **stubs** that log a warning and return empty values. Replace the method bodies in AwsSecretManagerOperations and GcpSecretManagerOperations before enabling those features in production.
- Nonce values are **never logged** — logging a nonce in plaintext would allow an attacker with log access to inject arbitrary inline scripts.
- The Server response header is masked to webserver to avoid exposing platform information.
- Review AllowSelfSignedCertificates = false (default) before deploying mTLS; self-signed certificates should only be used in development.
- AWS AccessKeyId and SecretAccessKey must **never** appear in appsettings.json — use User Secrets, environment variables, or IAM instance roles.
- For **AWS Cognito**, prefer IAM roles or Cognito Identity Pools over static credentials; never commit AppClientSecret to source control.
- GCP credentials should use **Application Default Credentials (ADC)** rather than committing service-account JSON files.
- For **GCP Identity**, the ClientSecret must be stored in User Secrets or environment variables — never in appsettings.json.

Files most relevant to these diagrams

- WebAppExperimental266/Program.cs
- WebAppExperimental266/Controllers/HomeController.cs
- WebAppExperimental266/Controllers/RecordsController.cs
- WebAppExperimental266/Controllers/AdminRecordsController.cs
- WebAppExperimental266/Controllers/GroupAdminRecordsController.cs
- WebAppExperimental266/Extensions/ServiceCollectionExtensions.cs
- WebAppExperimental266/Extensions/CrudAndAdminExtensions.cs
- WebAppExperimental266/Services/AdminCertificateRequirementHandler.cs

- [WebAppExperimental266/Services/GroupAdminCertificateRequirementHandler.cs](#)
- [WebAppExperimental266/Services/YubiKeyRequirementHandler.cs](#)
- [WebAppExperimental266/Services/CrudRecordAuthorizationHandler.cs](#)
- [WebAppExperimental266/Views/Shared/_Layout.cshtml](#)
- [WebAppExperimental266/Views/Home/Index.cshtml](#)
- [WebAppExperimental266/Views/Records/Index.cshtml](#)
- [WebAppExperimental266/Views/AdminRecords/Index.cshtml](#)
- [WebAppExperimental266/Views/GroupAdminRecords/Index.cshtml](#)
- [WebAppExperimental266.Tests/Integration/ApplicationIntegrationTests.cs](#)
- [WebAppExperimental266.Tests/Integration/MtlsIntegrationTests.cs](#)
- [docs/YUBIKEY_OPENBSD_ADMIN_GUIDE.md](#)

Generated from [docs/en-US/SITE_FEATURES_ARCHITECTURE_GUIDE.md](#) for repository documentation.